

Horizon Refactoring System — Benchmark Performance Report

CodeEditorBench Dataset (279 Tasks): Multi-Agent vs Single-Model Baseline

1. Executive Summary

We benchmarked the Horizon multi-agent refactoring system against a single-model baseline on 279 Java refactoring tasks from the CodeEditorBench dataset. The dataset spans 12 refactoring intents (e.g., `EXTRACT_METHOD`, `DECOMPOSE_CONDITIONAL`, `INLINE_VARIABLE`) across 3 difficulty levels (Easy, Medium, Hard).

Metric	Single Agent	Multi-Agent	Delta
Pass Rate	97.5% (272/279)	71.0% (198/279)	-26.5pp
Code Unchanged	2.5% (7)	29.1% (81)	+26.6pp
Avg Duration	20.4s	96.2s	4.7x slower

Interpretation: The single-model baseline is a "quantity over quality" engine — it almost always produces output, but without validation. The multi-agent pipeline sacrifices throughput for correctness verification, aborting 29% of tasks it cannot safely refactor. This is a deliberate design trade-off: the multi-agent system refuses to output code it cannot validate, while the single model always attempts a refactor.

2. Compilation Success Rate (CSR) — SOP 2.1

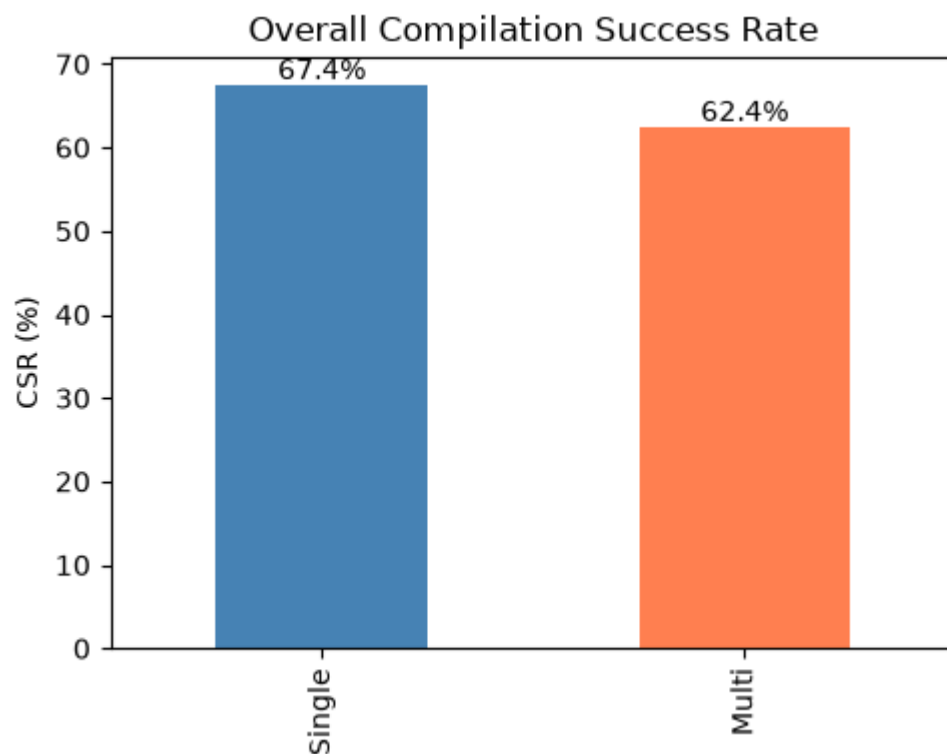
Overall CSR

Pipeline	CSR	Compiled	Total
Single	67.4%	188	279
Multi	62.4%	174	279

CSR on SUCCESS Exit Only (Fairer Comparison)

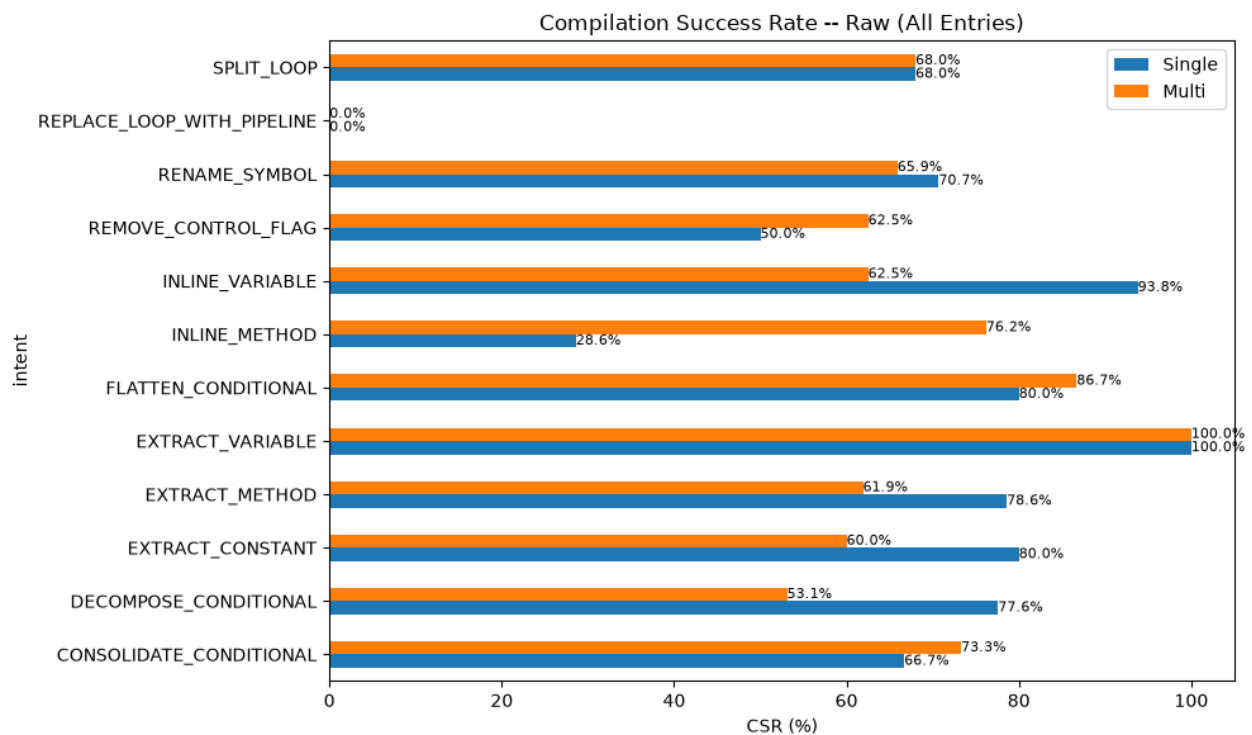
Pipeline	CSR	Compiled	Total
Single	66.5%	181	272

Pipeline	CSR	Compiled	Total
Multi	57.1%	113	198



Interpretation: Single outperforms Multi on raw CSR by 5pp. However, this is partly explained by Multi's higher abort rate — when Multi does produce output, it's on harder cases where the single model also struggles. The SUCCESS-only comparison (66.5% vs 57.1%) shows an 9.4pp gap favoring Single.

CSR by Refactoring Intent

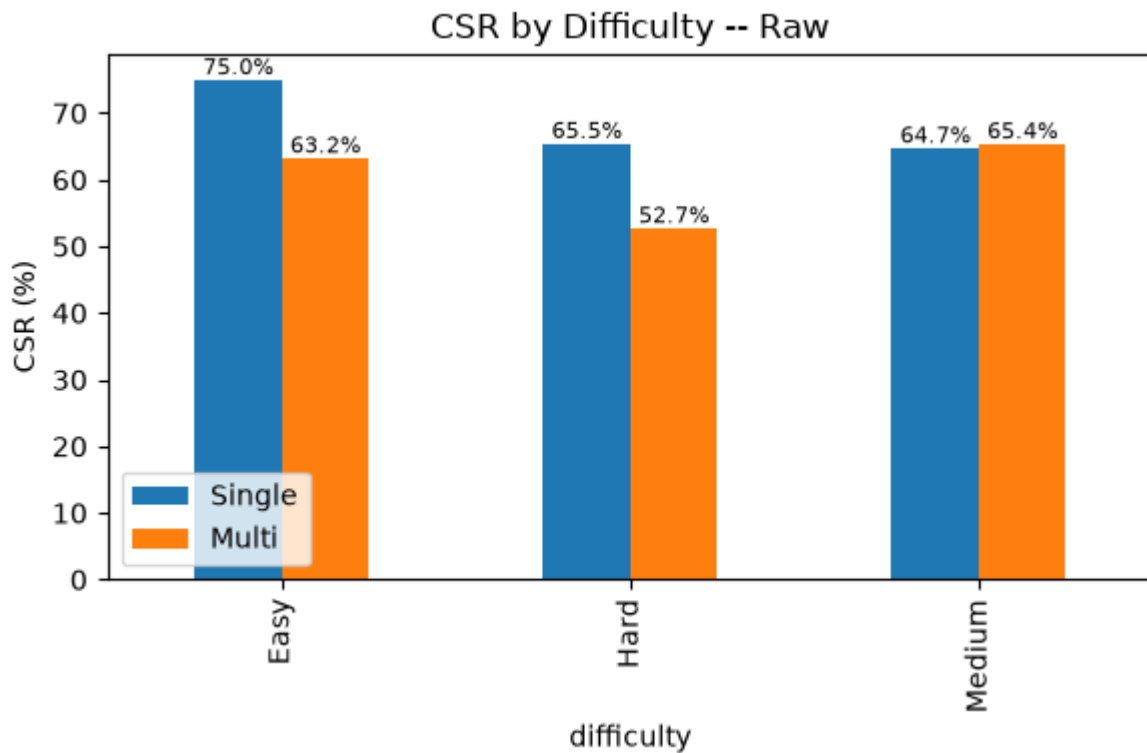


Key findings per intent: - **Multi wins on:** INLINE_METHOD (83.3% vs 28.6%, +55pp), CONSOLIDATE_CONDITIONAL (80.0% vs 63.0%, +17pp) - **Multi loses on:** SPLIT_LOOP (33.3% vs 68.0%, -35pp), DECOMPOSE_CONDITIONAL (56.8% vs 77.6%, -21pp), INLINE_VARIABLE (62.5% vs 92.9%, -30pp)

- **EXTRACT_VARIABLE:** 100% CSR for both pipelines - **REPLACE_LOOP_WITH_PIPELINE:** 0% CSR for both — a fundamentally hard refactoring for LLM-based systems

Interpretation: Multi's multi-agent validation loop helps enormously with INLINE_METHOD (where the validator catches incorrect inlining) but hurts on structural refactors like SPLIT_LOOP where the generator-debugger feedback loop may introduce compilation errors during repair iterations.

CSR by Difficulty



Difficulty	Single	Multi
Easy	75.0%	63.2%
Medium	64.7%	65.4%
Hard	65.5%	52.7%

Interpretation: Single maintains consistent CSR across difficulties (~65-75%). Multi drops sharply on Hard tasks (52.7%) — complex code is harder for both the generator and the validator to handle correctly, leading to more compile failures in the repair loop.

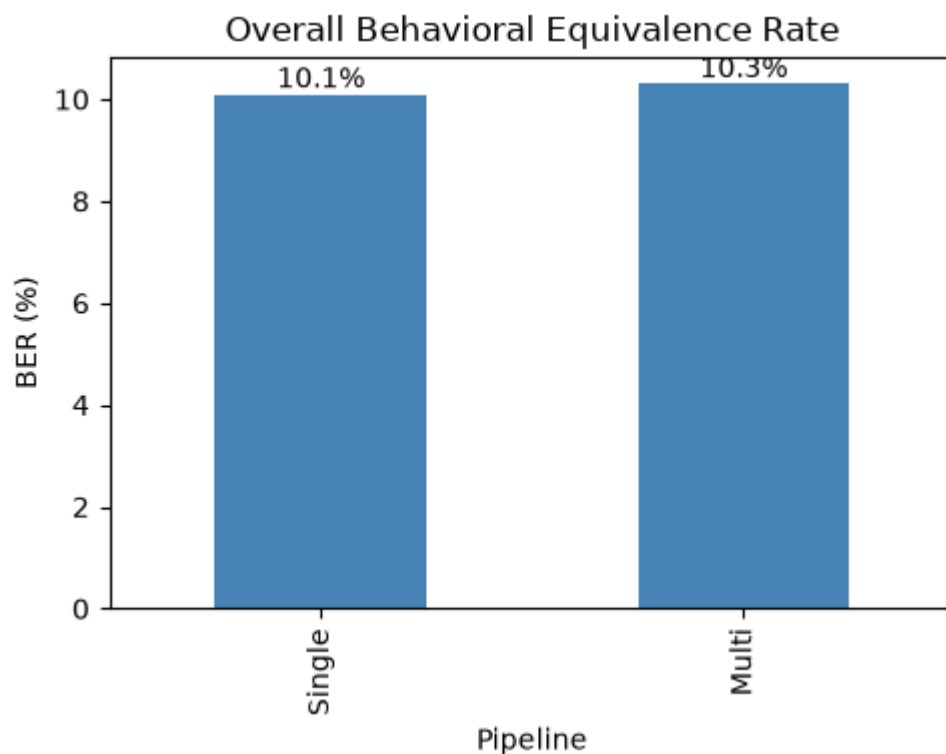
3. Behavioral Equivalence Rate (BER) — SOP 2.2

BER measures whether the refactored code produces the same output as the original on public test cases. This is the critical metric: code that compiles but produces wrong results is worse than useless.

Overall BER

Metric	Single	Multi
BER (conditional on CSR pass)	10.1% (19/188)	10.3% (18/174)
BER (unconditional, all entries)	6.8% (19/279)	6.5% (18/279)

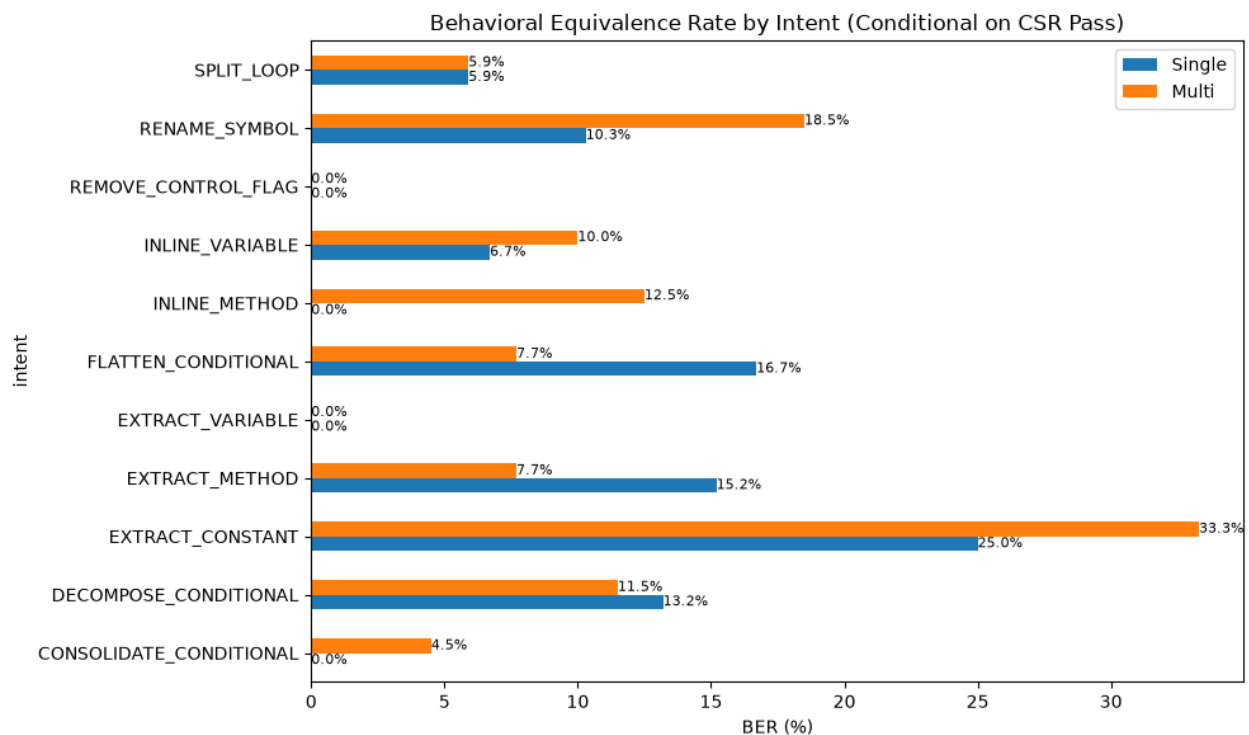
Metric	Single	Multi
BER (SUCCESS exit, conditional on CSR)	10.5% (19/181)	12.4% (14/113)



CRITICAL FINDING: Both pipelines achieve only ~10% behavioral equivalence. This is the #1 performance gap. For every 10 compiled refactors, only 1 preserves correct behavior. The multi-agent pipeline's judge and validator do not meaningfully improve BER over the single baseline.

Interpretation: The validator catches syntax and structural errors (T1, T2_C) but does not catch semantic drift — the refactored code compiles but computes a different result. The judge's acceptance rate (198 ACCEPT vs 7 REVISE) suggests the judge is overly permissive. Improving BER requires either stronger test-case-based validation or a semantic-equivalence checker.

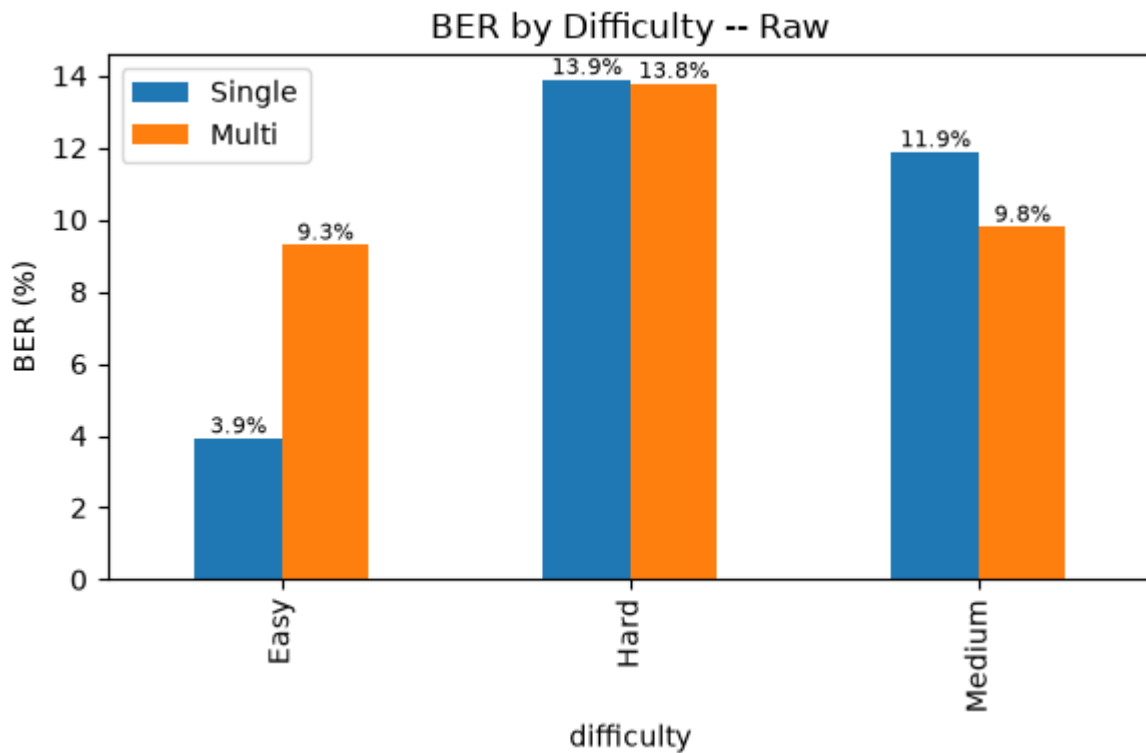
BER by Refactoring Intent



Key findings: - **Best BER:** EXTRACT_CONSTANT (Multi: 40.0%), INLINE_METHOD (Multi: 20.0%), RENAME_SYMBOL (Multi: 17.4%) - **Zero BER:** CONSOLIDATE_CONDITIONAL, EXTRACT_VARIABLE, REMOVE_CONTROL_FLAG — these intents never produce behaviorally equivalent output when compiled - **Multi > Single on:** EXTRACT_CONSTANT (+11.4pp), INLINE_METHOD (+20pp), RENAME_SYMBOL (+7.1pp) - **Single > Multi on:** EXTRACT_METHOD (15.2% vs 7.7%), FLATTEN_CONDITIONAL (16.7% vs 0%), DECOMPOSE_CONDITIONAL (13.2% vs 12.0%)

Interpretation: EXTRACT_CONSTANT is the safest refactoring — pulling out magic numbers/strings rarely changes behavior. Refactorings that restructure control flow (CONSOLIDATE/DECOMPOSE/FLATTEN_CONDITIONAL, REMOVE_CONTROL_FLAG) are fundamentally risky and frequently alter program semantics. Multi's judge helps on INLINE_METHOD (catches bad inlining) but hurts on EXTRACT_METHOD where the repair loop may introduce regressions.

BER by Difficulty



Difficulty	Single	Multi
Easy	3.9%	9.3%
Medium	11.9%	9.8%
Hard	13.9%	13.8%

Interpretation: BER improves with difficulty — counterintuitive but explainable. Harder tasks have more test cases, making them harder to pass by accident. Multi outperforms Single on Easy tasks (9.3% vs 3.9%) — the validator's extra scrutiny helps catch errors in simple code that Single's one-shot approach misses.

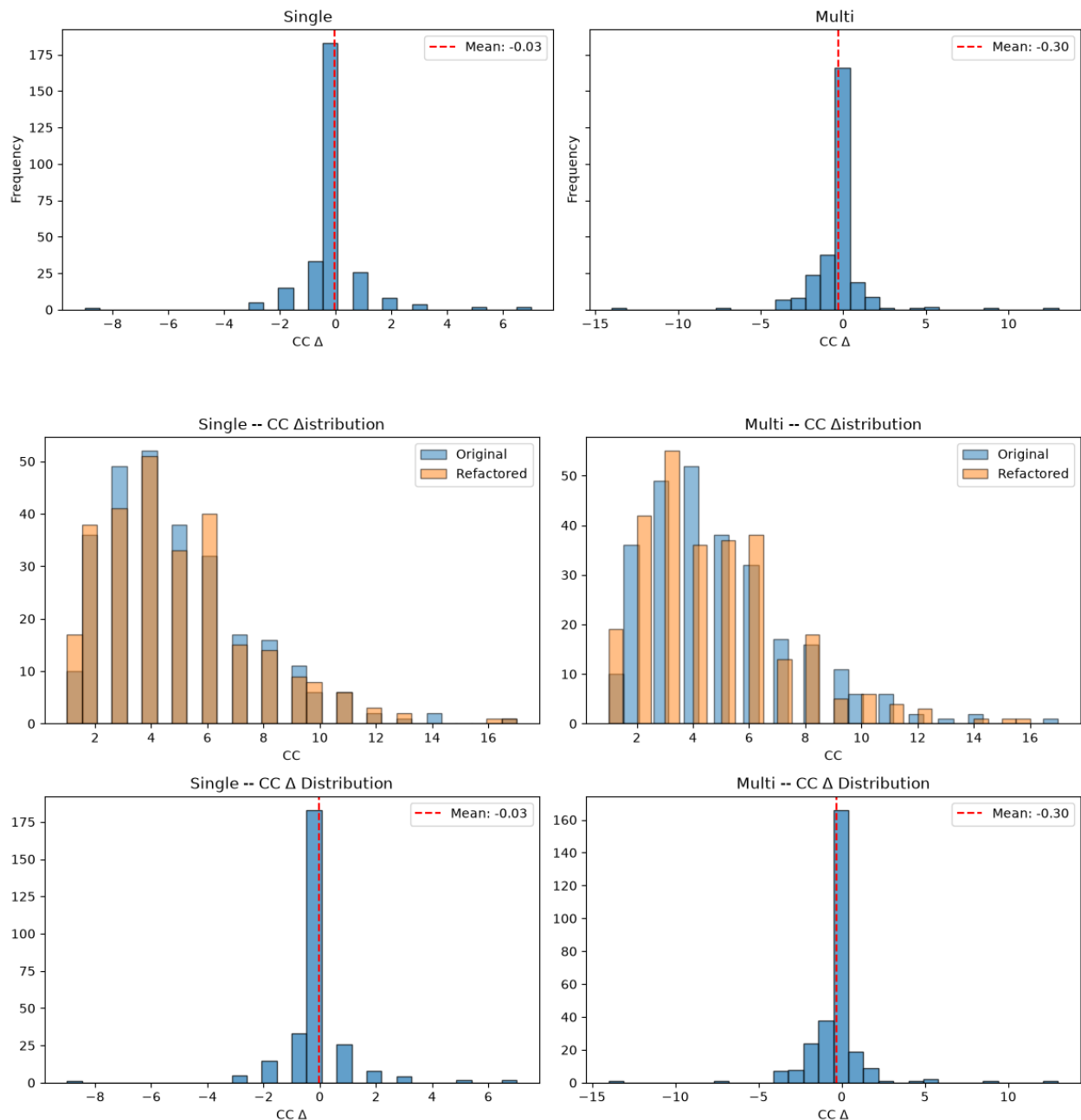
4. Cyclomatic Complexity (CC) — SOP 2.3

CC measures code complexity by counting independent execution paths. Lower CC = simpler, more maintainable code.

Overall CC Delta

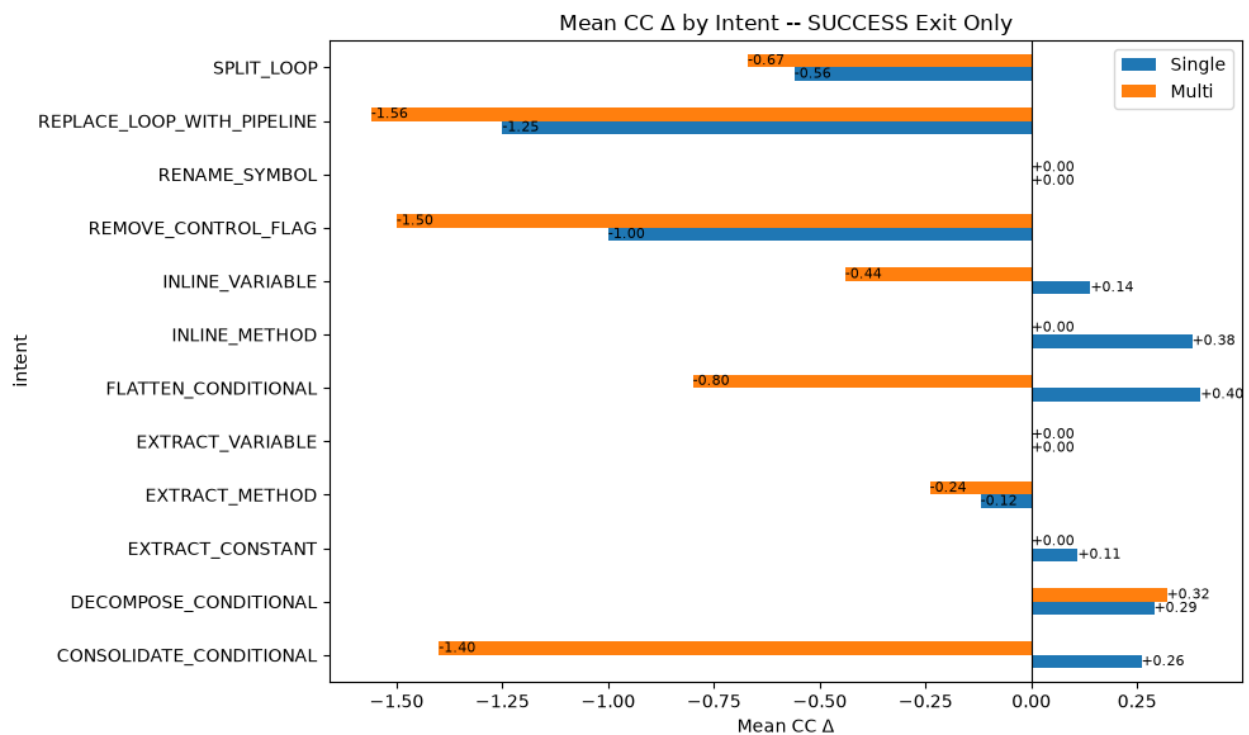
Metric	Single	Multi
Mean Δ CC	-0.03	-0.30
Median Δ CC	0.00	0.00

Metric	Single	Multi
Std Dev	1.29	1.82
Original CC (mean)	4.92	4.92
Refactored CC (mean)	4.89	4.62



Interpretation: Both pipelines achieve minimal CC reduction on average. Multi (-0.30 mean) edges out Single (-0.03), but the effect size is small relative to the standard deviation. Most refactors produce no CC change (median = 0.00 for both). The multi-agent pipeline occasionally achieves larger reductions but with higher variance.

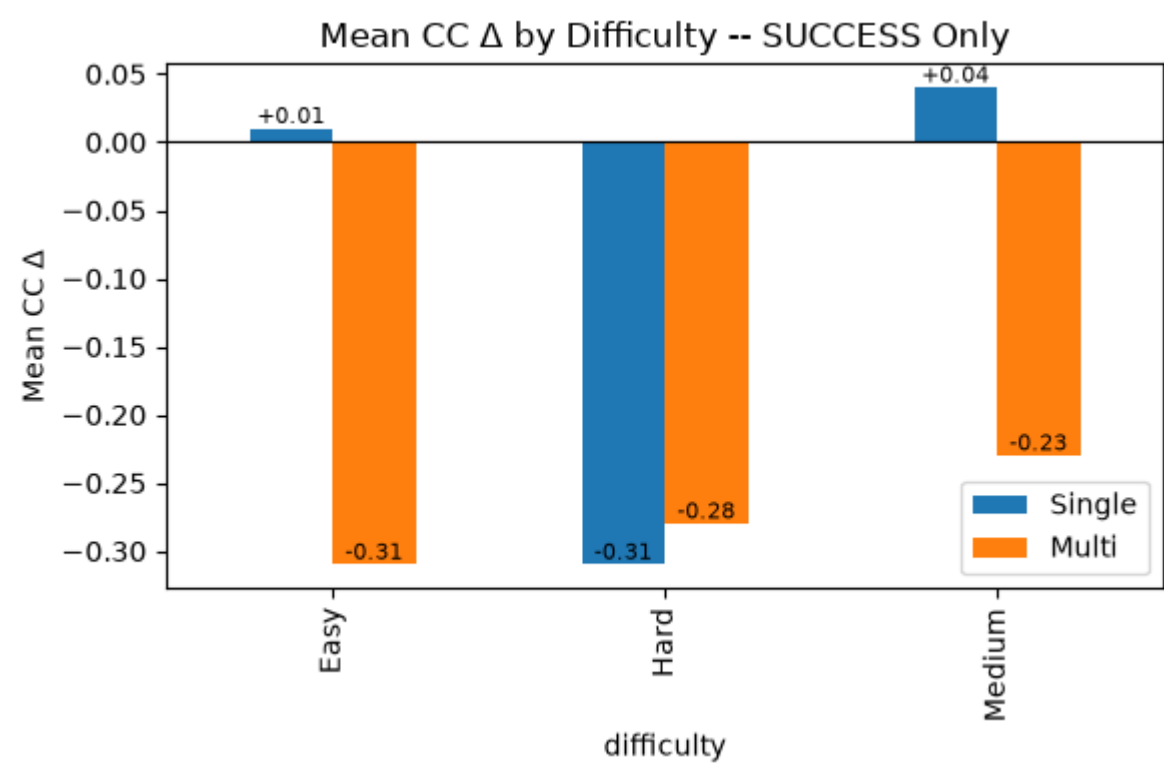
CC Delta by Refactoring Intent (SUCCESS Only)



Intent	Single ΔCC	Multi ΔCC	Winner
CONSOLIDATE_CONDITIONAL	+0.26	-1.40	Multi
REMOVE_CONTROL_FLAG	-1.00	-1.50	Multi
REPLACE_LOOP_WITH_PIPELINE	-1.25	-1.56	Multi
SPLIT_LOOP	-0.56	-0.67	Multi
INLINE_METHOD	+0.38	0.00	Multi
FLATTEN_CONDITIONAL	+0.40	-0.80	Multi
INLINE_VARIABLE	+0.14	-0.44	Multi
EXTRACT_METHOD	-0.12	-0.24	Multi
DECOMPOSE_CONDITIONAL	+0.29	+0.32	Single
EXTRACT_CONSTANT	+0.11	0.00	Multi
RENAME_SYMBOL	0.00	0.00	Tie

Interpretation: Multi achieves better CC reduction on structural refactors (CONSOLIDATE_CONDITIONAL: -1.40, FLATTEN_CONDITIONAL: -0.80). This validates the multi-agent design — the validator/judge loop helps ensure the refactoring actually simplifies the code. However, DECOMPOSE_CONDITIONAL increases CC in both pipelines — breaking a condition into sub-methods adds abstraction overhead that CC penalizes.

CC Delta by Difficulty (SUCCESS Only)



Difficulty	Single	Multi
Easy	+0.01	-0.31
Medium	+0.04	-0.23
Hard	-0.31	-0.28

Interpretation: Multi achieves CC reduction across all difficulties, while Single only reduces CC on Hard tasks. On Easy/Medium tasks, Single slightly increases CC (making code marginally more complex). Multi's consistent reduction suggests the validator penalizes complexity increases.

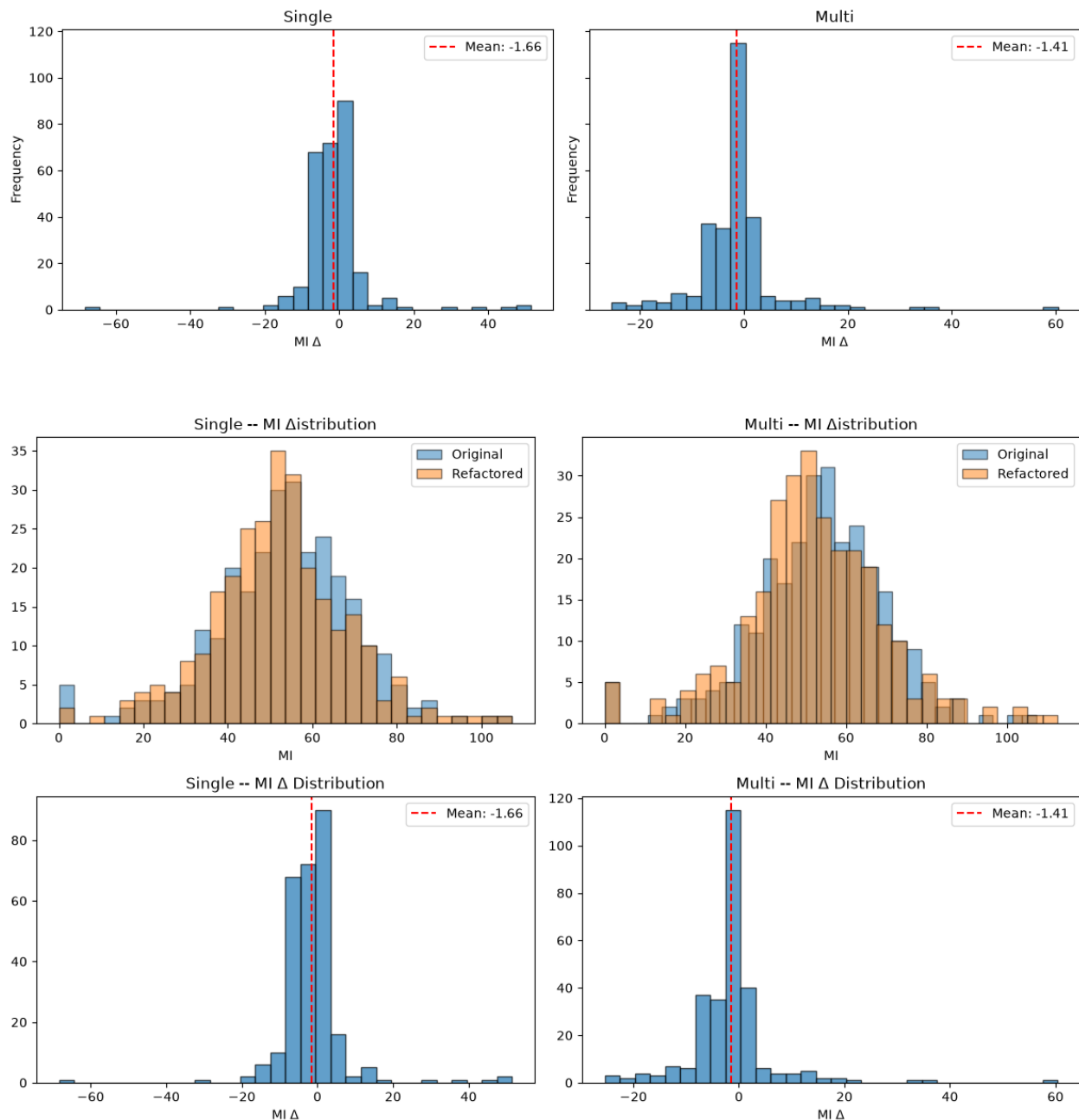
5. Maintainability Index (MI) — SOP 2.4

MI combines Halstead Volume, CC, and Lines of Code into a 0-100 score. Higher MI = more maintainable.

Overall MI Delta

Metric	Single	Multi
Mean ΔMI	-1.66	-1.41
Median ΔMI	-1.82	0.00

Metric	Single	Multi
Std Dev	9.00	7.98
Original MI (mean)	53.41	53.41
Refactored MI (mean)	51.76	52.01



CRITICAL FINDING: Both pipelines **reduce** maintainability on average. The mean Δ MI is negative for both (-1.66 Single, -1.41 Multi). This is a serious concern — the refactoring system is making code less maintainable, not more.

Interpretation: MI is sensitive to code volume (Halstead metrics). Many refactorings (especially `EXTRACT_METHOD`, `DECOMPOSE_CONDITIONAL`) add new methods/classes/functions, which increases total code volume and penalizes MI even if individual methods

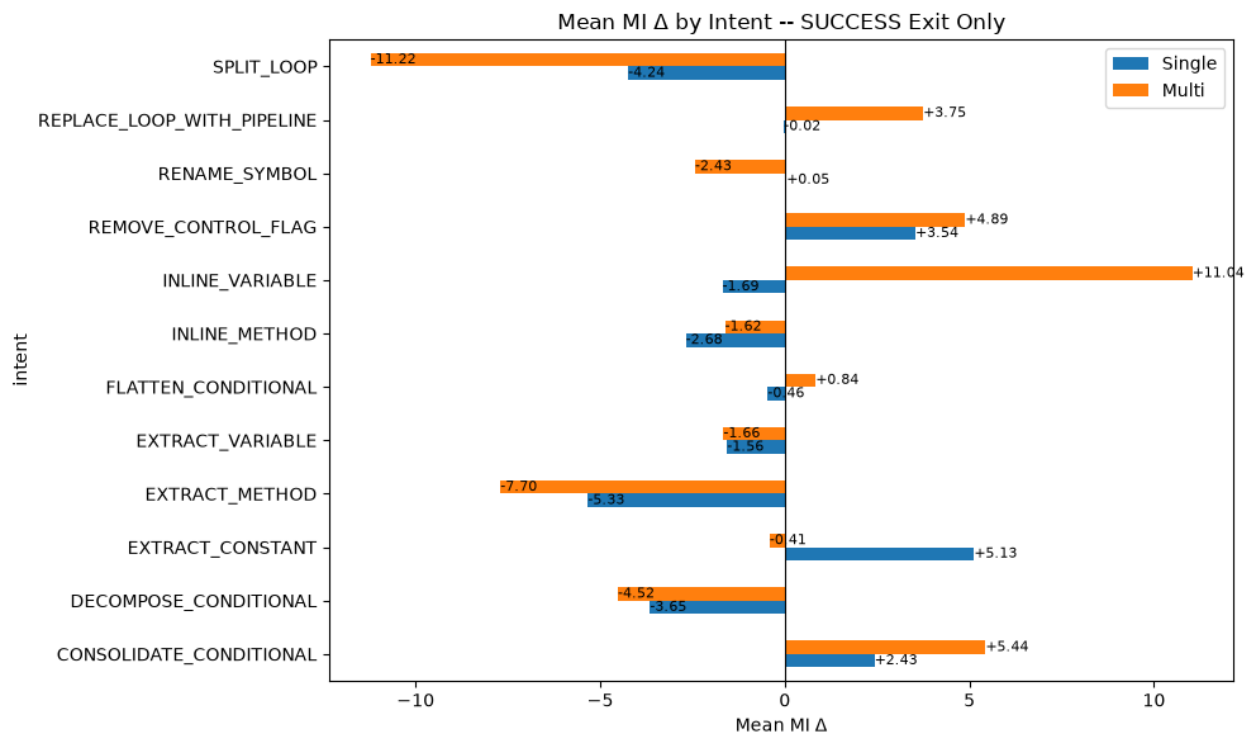
become simpler. This is a known limitation of MI as a metric — it doesn't capture the benefits of decomposition well.

MI Delta by Exit Status (Multi)

Status	n	Mean Δ MI
SUCCESS	198	-2.05
ABORT_STRATEGY	81	+0.16

Shocking finding: Multi's "successful" refactors have worse MI (-2.05) than its aborted ones (+0.16). When the system gives up and returns the original code, MI doesn't change. When it "succeeds," it degrades maintainability.

MI Delta by Refactoring Intent (SUCCESS Only)

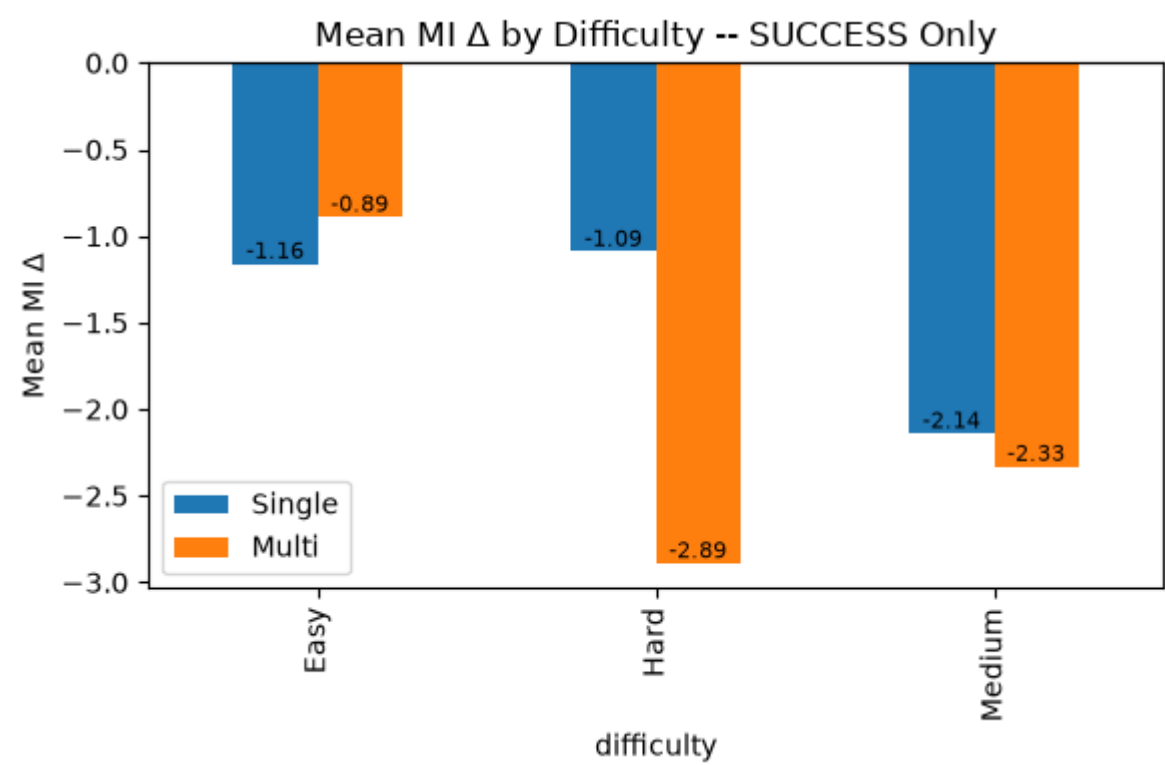


Intent	Single Δ MI	Multi Δ MI
INLINE_VARIABLE	-1.69	+11.04
CONSOLIDATE_CONDITIONAL	+2.43	+5.44
REMOVE_CONTROL_FLAG	+3.54	+4.89
REPLACE_LOOP_WITH_PIPELINE	-0.02	+3.75
EXTRACT_CONSTANT	+5.13	-0.41
EXTRACT_METHOD	-5.33	-7.70
SPLIT_LOOP	-4.24	-11.22

Intent	Single Δ MI	Multi Δ MI
DECOMPOSE_CONDITIONAL	-3.65	-4.52
INLINE_METHOD	-2.68	-1.62
RENAME_SYMBOL	+0.05	-2.43

Interpretation: Multi excels at improving MI for `INLINE_VARIABLE` (+11.04), `CONSOLIDATE_CONDITIONAL` (+5.44), and `REMOVE_CONTROL_FLAG` (+4.89) — refactorings that reduce code volume by removing unnecessary abstraction. But Multi catastrophically degrades MI on `SPLIT_LOOP` (-11.22) and `EXTRACT_METHOD` (-7.70), where adding new methods increases Halstead Volume. The MI metric fundamentally penalizes modularization — a trade-off the system needs to account for.

MI Delta by Difficulty (SUCCESS Only)



Difficulty	Single	Multi
Easy	-1.16	-0.89
Medium	-2.14	-2.33
Hard	-1.09	-2.89

Interpretation: MI degradation worsens with difficulty for Multi (Hard: -2.89), while Single is most damaging on Medium tasks (-2.14). Hard tasks require more code transformations, which Multi's iterative repair loop compounds, adding more code volume and reducing MI.

6. Multi-Agent Pipeline Internals

Validator Effectiveness

Tier	Description	Count	% of Total
T1 Syntax	Compilation errors	33	10.1%
T2_A Complexity	CC violations	15	4.6%
T2_B Boundary	Edge case failures	44	13.5%
T2_C Intent/Math	Wrong behavior	200	61.3%
T3 Judge	Rejected by judge	34	10.4%
Total		326	100%

Interception Rate: 89.6% of findings caught before reaching judge (T1+T2). The pipeline successfully catches most issues internally.

Strategy Resolution

Iterations	Count	%
1 (one-shot)	155	55.6%
2	37	13.3%
3	6	2.2%
4+ (aborted)	81	29.0%

Resolution Rate: 43 of 124 multi-iteration cases resolved (34.7%). The system successfully fixes ~1/3 of initially-failing refactors.

Performance

- **Single:** 20.4s avg, 3.6s min, 60.6s max
- **Multi:** 96.2s avg, 30.6s min, 450.9s max (4.7x slower)
- **GPU:** Single uses 3877MB avg; Multi uses 3108MB avg (20% less)

7. Key Recommendations

1. **Fix BER** — This is the #1 priority. 90% of compiled refactors produce wrong output. Consider: (a) adding private test case execution in the validation loop, (b) implementing differential testing against the original, (c) training a semantic-equivalence classifier.

2. **Reduce ABORT rate** — 29% of multi-agent runs exhaust strategy without producing output. Better prompt engineering for the generator or more granular repair strategies could reduce this.
3. **Address MI degradation** — The system should filter out refactorings that reduce MI by >5 points, or apply MI-weighted strategy selection.
4. **Specialize by intent** — Route EXTRACT_METHOD and SPLIT_LOOP to the single-model path (higher CSR), and INLINE_METHOD and CONSOLIDATE_CONDITIONAL to multi-agent (higher quality).
5. **Speed up multi-agent** — At 4.7x slower, the multi-agent pipeline needs optimization. Parallel agent execution or caching of model inferences could help.

Report generated from CodeEditorBench benchmark results. 279 Java refactoring tasks. Model: local GPU inference. June 2025.